

PDFShell

Architecture & Technology Stack Specification

PDFShell is a fully open-source, browser-based PDF toolkit. It covers six core tools — Edit, OCR (text extraction), Compress, Merge, Split, and Convert — with a privacy-first design: no files ever leave the user's device. All processing runs inside the browser via WebAssembly and WebWorkers.

Privacy-First Architecture — All PDF processing happens entirely in the browser via WebAssembly and WebWorkers. Files are never uploaded to a server, never stored remotely, and never seen by third parties. This is PDFShell's single biggest differentiator from Smallpdf, ilovepdf, and every comparable tool.

The six tools

 Edit Add text, images, and annotations. Modify metadata. Fill form fields. Powered by pdf-lib.	 OCR Extract text from scanned PDFs. 100+ languages. Runs Tesseract.js v6 entirely in-browser.	 Compress Reduce file size using Ghostscript compiled to WebAssembly. Presets: screen / ebook / print.
 Merge Combine multiple PDFs into one. Drag-and-drop page reordering. Powered by pdf-lib.	 Split Extract pages or page ranges into separate files. Batch export supported.	 Convert DOCX → PDF via mammoth.js + jsPDF. Images (JPG, PNG) → PDF via pdf-lib. Client-side only.

UI Layer

Library / Tool	Purpose	Notes
Next.js 15	App Router + React Server Components. Static export to Vercel.	Edge-deployable. No backend required.
shadcn/ui	Copy-paste component library built on Radix UI primitives.	Top pick for 2026. Zero runtime overhead. Fully owned code.
Tailwind CSS 4	Utility-first styling. Dark mode automatic.	Required for shadcn/ui. Best performance/bundle tradeoff.

Framer Motion	Page transitions, drag-and-drop page thumbnail reordering.	Essential for the Merge/Split UX.
----------------------	--	-----------------------------------

State Management & Developer Experience

Library / Tool	Purpose	Notes
TypeScript	Full type safety across all packages and the web app.	Mandatory. Catches integration errors between PDF libs and UI.
Zustand	Lightweight global state (active tool, file list, processing status).	Far simpler than Redux for this use case.
React Dropzone	File upload UX: drag-and-drop zone with accept/reject feedback.	Handles multi-file uploads, MIME filtering, and progress.
Vitest + Playwright	Unit tests for pdf-core / ocr-engine packages. E2E tests for each tool.	Vitest for fast unit tests; Playwright for browser automation.

PDF Core Libraries (Browser-Native via WASM / JS)

Library / Tool	Purpose	Notes
pdf-lib	Merge, split, edit: add text/images, fill forms, modify metadata.	Works in browser + Node with no native deps. Clean TypeScript API.
PDF.js (Mozilla)	Render PDF pages onto HTML canvas for preview and thumbnails.	Industry standard. Also feeds pages into the Tesseract OCR pipeline.
Ghostscript WASM	PDF compression. Compiled from C to WebAssembly via Emscripten.	Run inside a WebWorker. Presets: /screen /ebook /printer /prepress.

Compression preset guide: /screen (72 dpi, smallest) → /ebook (150 dpi, balanced) → /printer (300 dpi, high quality) → /prepress (300 dpi + colour preservation). Recommend **/ebook** as the default — best balance of size reduction and visual quality.

OCR & Conversion Libraries

Library / Tool	Purpose	Notes
Tesseract.js v6	Client-side OCR. 100+ languages. Improved memory management in v6.	Pipeline: PDF.js renders page → canvas image → Tesseract recognises text.
jsPDF	Lightweight client-side PDF generation from HTML or images.	Used in the Convert tool for image → PDF and HTML → PDF export.
mammoth.js	Converts DOCX to clean semantic HTML.	Feeds into jsPDF for DOCX → PDF conversion. No server required.

OCR accuracy notes: Below 150 DPI input quality, recognition drops to 70–80%. Multi-column layouts and tables can produce scrambled output. Handwriting is out of scope. Communicate these limits clearly in the tool UI to set user expectations.

Infrastructure & Deployment		
Library / Tool	Purpose	Notes
Vercel	Static site hosting with edge CDN. Zero server costs.	Entire app deploys as a static export — no backend, no database.
pnpm + Turborepo	Monorepo package management with build caching.	Manages apps/web + packages/pdf-core, ocr-engine, compress-engine.
GitHub Actions	CI/CD: type-check, test, build, deploy on every pull request.	Vitest unit tests + Playwright E2E in the pipeline.
Sentry	Error and performance monitoring in production.	Lightweight SDK. Catches WASM failures and OCR timeouts.

Monorepo project structure



The monorepo separates concerns cleanly: pdf-core, ocr-engine, and compress-engine are standalone packages that can be tested in isolation and potentially published to npm as open-source libraries independently of the web app.

Development roadmap

Phase 1	MVP — Merge & Split	pdf-lib + drag-and-drop page grid. Demonstrates privacy-first value instantly.
Phase 2	Compress + Preview	Ghostscript WASM (WebWorker) + PDF.js renderer + thumbnail strip.
Phase 3	OCR + Convert	Tesseract.js pipeline (PDF.js canvas → OCR). mammoth.js + jsPDF for DOCX conversion.
Phase 4	Polish + PWA	Dark mode (Tailwind auto), offline support, page reorder UX, Sentry monitoring.

Build Phase 1 first. Merge and Split are the simplest tools to implement with pdf-lib, and they immediately demonstrate the privacy-first value proposition. Each subsequent phase adds one WASM engine (Ghostscript, then Tesseract) — isolating complexity and making debugging straightforward.

Key differentiators

Zero uploads	Files never leave the browser. No server, no storage, no privacy risk.
Zero cost to run	Entire app is a static site on Vercel. No server compute costs ever.
Open source	Full codebase on GitHub. Community can audit, fork, and contribute.
No account needed	No signup, no login, no cookies beyond preferences. Just open and use.
Offline capable	Phase 4 adds PWA support. Works without an internet connection.
100+ OCR languages	Tesseract.js supports over 100 languages including local African scripts.

This specification is the reference document for the PDFShell open-source project. All library choices prioritise browser-native execution, TypeScript compatibility, active maintenance, and zero cost at runtime. The architecture is intentionally designed so that the entire application can be deployed and operated with no server infrastructure and no ongoing hosting costs beyond a free Vercel tier.